

```

/* Raquel del Valle Olivares y Xiya Ye Grupo 503 */
/* 100522214@alumnos.uc3m.es 100522159@alumnos.uc3m.es */

%{
    // SECCION 1 Declaraciones de C-Yacc

#include <stdio.h>
#include <ctype.h>           // declaraciones para tolower
#include <string.h>          // declaraciones para cadenas
#include <stdlib.h>          // declaraciones para exit ()

#define FF fflush(stdout);   // para forzar la impresion inmediata

int yylex () ;
int yyerror () ;
char *mi_malloc (int) ;
char *gen_code (char *) ;
char *int_to_string (int) ;
char *char_to_string (char) ;

void add_local_var(char *name);
int is_local_var(char *name);
char *get_var_name(char *name);

void add_param(char *name);
int is_param(char *name);
void limpia_params();

char temp [2048] ;
char funcion_actual[256] = "main";      // Almacenamos la función función actual
void limpia_vars_locales();

```

```
// Abstract Syntax Tree (AST) Node Structure
```

```
typedef struct ASTnode t_node ;
```

```
struct ASTnode {  
    char *op ;  
    int type ;           // leaf, unary or binary nodes  
    t_node *left ;  
    t_node *right ;  
} ;
```

```
// Definitions for explicit attributes
```

```
typedef struct s_attr {  
    int value ;          // - Numeric value of a NUMBER  
    char *code ;         // - to pass IDENTIFIER names, and other translations  
    t_node *node ;       // - for possible future use of AST  
} t_attr ;
```

```
#define YYSTYPE t_attr
```

```
%}
```

```
// Definitions for explicit attributes
```

```
%token NUMBER  
%token IDENTIF      // Identificador=variable  
%token INTEGER      // identifica el tipo entero  
%token STRING  
%token MAIN          // identifica el comienzo del proc. main
```

```

%token WHILE          // identifica el bucle main
%token PUTS           // identifica la impresión de cadenas
%token PRINTF         // identifica la impresión de cadenas y expresiones
%token IF             // identifica la condición if
%token ELSE           // identifica else
%token AND             // identifica &&
%token OR             // identifica ||
%token EQ             // identifica ==
%token NEQ            // identifica !=
%token LEQ            // identifica <=
%token GEQ            // identifica >=
%token FOR            // identifica el bucle for
%token INC            // identifica incremento
%token DEC            // identifica decremento
%token SWITCH         // identifica switch
%token CASE           // identifica un caso
%token DEFAULT        // identifica el caso por defecto
%token BREAK          // identifica el salto de bloque
%token VOID           // identifica una funcion
%token RETURN         // identifica el return

```

```

%right '='            // es la ultima operacion que se debe realizar (nivel 14)
%left OR              // nivel 12 de la tabla (||)
%left AND             // nivel 11 (&&)
%left EQ NEQ          // nivel 7 (== , !=)
%left '<' '>' LEQ GEQ  // nivel 6 (< , > , <= , >=)
%left '+' '-'         // nivel 4
%left '*' '/' '%'     // nivel 3
%right '!' UNARY_SIGN // mayor orden de precedencia, NOT y signos (nivel 2)

```

```
// Seccion 3 Gramatica - Semantico

axioma:      declaracion_vars axioma      { ; }
            | funcion_axioma              { ; }
            | funcion_main                 { ; }
            ;

funcion:      nombre_funcion '(' parametros_def ')' '{' declaraciones_locales cuerpo_funcion}' {
                sprintf (temp, "(defun %s (%s)%s\n%s\n)", $1.code, $3.code, $6.code, $7.code) ;
                printf("%s\n", temp) ;
                $$ .code = gen_code (temp) ; }

;

funcion_main:
        cabecera_main '{' declaraciones_locales lista_sentencias '}' { sprintf (temp, "(defun main ()%s\n%s\n)",
$3.code, $4.code) ;

                                printf("%s\n", temp) ;
                                $$ .code = gen_code (temp) ; }

;

declaracion_vars:
        INTEGER lista ';'    { printf("%s\n", $2.code);
                               $$=$2; }

;

declaraciones_locales:
                                { $$ .code = gen_code(""); } // lambda
|   declaraciones_locales declaracion_local { if (strlen($1.code) == 0) {
                                                $$ .code = $2.code;
                                            } else {
```

```

                                                                    sprintf (temp, "%s\n%s", $1.code, $2.code) ;
                                                                    $$$.code = gen_code(temp);
                                                                    } }

;

declaracion_local:
    INTEGER lista_local ';'          { $$ = $2 ; }
;

lista_local:  IDENTIF                { add_local_var($1.code);
                                      sprintf (temp, "(setq %s_%s 0)", funcion_actual, $1.code);
                                      $$$.code = gen_code(temp); }
    |  IDENTIF '=' NUMBER            { add_local_var($1.code);
                                      sprintf (temp, "(setq %s_%s %d)", funcion_actual, $1.code,
$3.value);
                                      $$$.code = gen_code(temp); }
    |  lista_local ',' IDENTIF        { add_local_var($3.code);
                                      sprintf (temp, "%s\n(setq %s_%s 0)", $1.code, funcion_actual,
$3.code);
                                      $$$.code = gen_code(temp); }
    |  lista_local ',' IDENTIF '=' NUMBER { add_local_var($3.code);
                                      sprintf (temp, "%s\n(setq %s_%s %d)", $1.code, funcion_actual,
$3.code, $5.value);
                                      $$$.code = gen_code(temp); }
    |  IDENTIF '[' NUMBER ']'        { add_local_var($1.code);
                                      sprintf (temp, "(setq %s_%s (make-array %d))", funcion_actual,
$1.code, $3.value);
                                      $$$.code = gen_code(temp); }
    |  lista_local ',' IDENTIF '[' NUMBER ']' { add_local_var($3.code);
                                      sprintf (temp, "%s\n(setq %s_%s (make-array %d))", $1.code,
funcion_actual, $3.code, $5.value);

```

```

                                $$$.code = gen_code(temp); }

;

sentencia:      IDENTIF '=' expresion ';'          { sprintf (temp, "(setf %s %s)", get_var_name($1.code),
$3.code) ;

                                $$$.code = gen_code (temp) ; }
| '@' expresion          { sprintf (temp, "(print %s)", $2.code) ;
                                $$$.code = gen_code (temp) ; }
| PUTS '(' STRING ')' ';' { sprintf (temp, "(print \"%s\")", $3.code) ;
                                $$$.code = gen_code (temp) ; }
| PRINTF '(' STRING lista_printf ')' ';'          { $$$.code = $4.code ; }
| WHILE '(' expresion ')' '{' lista_sentencias_bloque '}' { sprintf (temp, "(loop while %s do\n%s)",
$3.code, $6.code) ;

                                $$$.code = gen_code (temp) ; }

| IF '(' expresion ')' '{' lista_sentencias_bloque '}' { sprintf (temp, "(if %s\n (progn %s)\n)",
$3.code, $6.code) ;

                                $$$.code = gen_code (temp) ; }

| IF '(' expresion ')' '{' lista_sentencias_bloque '}'
  ELSE '{' lista_sentencias_bloque '}'
  %s)\n", $3.code, $6.code, $10.code) ;

                                { sprintf (temp, "(if %s\n (progn %s)\n (progn
                                $$$.code = gen_code (temp) ; }

| FOR '(' IDENTIF '=' expresion ';' expresion ';' INC '(' IDENTIF ')' ')' '{' lista_sentencias_bloque '}'
                                {sprintf (temp, "(setf %s %s)\n(loop while %s
do\n%s\n(setf %s (+ %s 1)))",
                                get_var_name($3.code), $5.code, $7.code, $15.code,
                                $$$.code = gen_code (temp); }
                                {sprintf (temp, "(setf %s %s)\n(loop while %s
do\n%s\n(setf %s (- %s 1)))",

```

```

get_var_name($11.code), get_var_name($11.code));

| SWITCH '(' IDENTIF ')' '{' lista_casos '}'
$6.code);

| SWITCH '(' IDENTIF ')' '{' lista_casos DEFAULT ':' lista_sentencias_bloque BREAK ';' '}'
get_var_name($3.code), $6.code, $9.code);

| IDENTIF '(' parametros_llamada ')' ';'

| IDENTIF '[' expresion ']' '=' expresion ';';
get_var_name($1.code), $3.code, $6.code);

;

lista_printf:  ',' elem_printf      { $$code = $2.code; }
| lista_printf ',' elem_printf      { sprintf (temp, "%s\n%s", $1.code, $3.code) ;
                                     $$code = gen_code (temp) ; }

;

elem_printf:  expresion      { sprintf (temp, "(princ %s)", $1.code) ;
                              $$code = gen_code (temp) ; }
| STRING      { sprintf (temp, "(princ \"%s\")", $1.code) ;
               $$code = gen_code (temp) ; }

;

```

```

get_var_name($3.code), $5.code, $7.code, $15.code,

$$code = gen_code (temp); }
{sprintf(temp, "(case %s\n%s)", get_var_name($3.code),

$$code = gen_code (temp) ; }
{sprintf(temp, "(case %s\n%s\n(otherwise (progn %s)))",

$$code = gen_code (temp) ; }
{ if (strlen($3.code) == 0) {
    sprintf(temp, "(%s)", $1.code);
} else {
    sprintf(temp, "(%s %s)", $1.code, $3.code);
}
$$code = gen_code (temp); }
{ sprintf (temp, "(setf (aref %s %s) %s)",

$$code = gen_code (temp) ; }

```

[illegible]

;

nombre_funcion:

```
IDENTIF      { strcpy(funcion_actual, $1.code);
               limpia_vars_locales();
               limpia_params();
               $$code = $1.code; }
```

;

```
cuerpo_funcion: lista_sentencias RETURN expresion ';' { sprintf(temp, "%s\n%s", $1.code, $3.code);
                                                         $$code = gen_code(temp); }
               | RETURN expresion ';'                  { $$code = $2.code; }
               | lista_sentencias                      { $$code = $1.code; }
               ;
```

cabecera_main:

```
MAIN '(' ')' { strcpy(funcion_actual, "main");
               limpia_vars_locales(); }
```

;

parametros_def:

```
/* lambda */      { $$code = gen_code(""); }
| lista_param_def  { $$ = $1; }
;
```

lista_param_def:

```
INTEGER IDENTIF { add_param($2.code);
                  $$code = $2.code; }
| lista_param_def ',' INTEGER IDENTIF { add_param($4.code);
                                         sprintf(temp, "%s %s", $1.code, $4.code);
```

```

                                $$$.code = gen_code(temp); }

;

parametros_llamada:
    /* lambda */                { $$$.code = gen_code(""); }
    | lista_param_llamada       { $$ = $1; }
;

lista_param_llamada:
    expresion                   { $$ = $1; }
    | lista_param_llamada ',' expresion { sprintf(temp, "%s %s", $1.code, $3.code);
                                         $$$.code = gen_code(temp); }
;

lista_casos:
    caso                        { $$ = $1; }
    | lista_casos caso          { sprintf(temp, "%s\n%s", $1.code, $2.code);
                                $$$.code = gen_code(temp); }
;

caso:
    CASE NUMBER ':' lista_sentencias_bloque BREAK ';' { sprintf(temp, "(%d(progn %s))", $2.value, $4.code);
                                                         $$$.code = gen_code(temp); }
;

expresion:
    termino                    { $$ = $1 ; }
    | expresion '+' expresion   { sprintf (temp, "(+ %s %s)", $1.code, $3.code) ;
                                $$$.code = gen_code (temp) ; }
    | expresion '-' expresion   { sprintf (temp, "(- %s %s)", $1.code, $3.code) ;
                                $$$.code = gen_code (temp) ; }
    | expresion '*' expresion   { sprintf (temp, "(* %s %s)", $1.code, $3.code) ;
                                $$$.code = gen_code (temp) ; }
    | expresion '/' expresion   { sprintf (temp, "(/ %s %s)", $1.code, $3.code) ;

```

```

    $.code = gen_code (temp) ; }
|  expresion '%' expresion    { sprintf (temp, "(mod %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion AND expresion    { sprintf (temp, "(and %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion OR expresion     { sprintf (temp, "(or %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion EQ expresion     { sprintf (temp, "(= %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion NEQ expresion    { sprintf (temp, "(/= %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion '<' expresion     { sprintf (temp, "< %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion '>' expresion     { sprintf (temp, "> %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion LEQ expresion    { sprintf (temp, "(<= %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
|  expresion GEQ expresion    { sprintf (temp, "(>= %s %s)", $1.code, $3.code) ;
    $.code = gen_code (temp) ; }
;

```

```

termino:      operando      { $$ = $1 ; }
|  '+' operando %prec UNARY_SIGN { $$ = $1 ; }
|  '-' operando %prec UNARY_SIGN { sprintf (temp, "(- %s)", $2.code) ;
    $.code = gen_code (temp) ; }
|  '!' operando %prec UNARY_SIGN { sprintf (temp, "(not %s)", $2.code) ;
    $.code = gen_code (temp) ; }
;

```

```

operando:      IDENTIF      { sprintf (temp, "%s", get_var_name($1.code)) ;
    $.code = gen_code (temp) ; }

```

```

|    NUMBER                                { sprintf (temp, "%d", $1.value) ;
                                           $$ .code = gen_code (temp) ; }
|    '(' expresion ')'                    { $$ = $2 ; }
|    IDENTIF '(' parametros_llamada ')'   { if (strlen($3.code) == 0) {
                                           sprintf(temp, "(%s)", $1.code);
                                           } else {
                                           sprintf(temp, "(%s %s)", $1.code, $3.code);
                                           }
                                           $$ .code = gen_code (temp); }
|    IDENTIF '[' expresion ']'            { sprintf (temp, "(aref %s %s)", get_var_name($1.code), $3.code) ;
                                           $$ .code = gen_code (temp) ; }
;

```

%% // SECCION 4 Codigo en C

```
int n_line = 1 ;
```

```
int yyerror (mensaje)
```

```
char *mensaje ;
```

```
{
```

```
    fprintf (stderr, "%s en la linea %d\n", mensaje, n_line) ;
```

```
    printf ( "\n" ) ;      // bye
```

```
}
```

```
char *int_to_string (int n)
```

```
{
```

```
    char ltemp [2048] ;
```

```
    sprintf (ltemp, "%d", n) ;
```

```
    return gen_code (ltemp) ;  
}
```

```
char *char_to_string (char c)  
{  
    char ltemp [2048] ;  
  
    sprintf (ltemp, "%c", c) ;  
  
    return gen_code (ltemp) ;  
}
```

```
char *mi_malloc (int nbytes)          // reserva n bytes de memoria dinamica  
{  
    char *p ;  
    static long int nb = 0;           // sirven para contabilizar la memoria  
    static int nv = 0 ;               // solicitada en total  
  
    p = malloc (nbytes) ;  
    if (p == NULL) {  
        fprintf (stderr, "No queda memoria para %d bytes mas\n", nbytes) ;  
        fprintf (stderr, "Reservados %ld bytes en %d llamadas\n", nb, nv) ;  
        exit (0) ;  
    }  
    nb += (long) nbytes ;  
    nv++ ;  
  
    return p ;  
}
```

```

/*****
/***** Seccion de Palabras Reservadas *****/
/*****/

```

```

typedef struct s_keyword { // para las palabras reservadas de C
    char *name ;
    int token ;
} t_keyword ;

```

```

t_keyword keywords [] = { // define las palabras reservadas y los
    "main",          MAIN,          // y los token asociados
    "int",           INTEGER,
    "puts",          PUTS,
    "printf",        PRINTF,
    "while",         WHILE,
    "if",            IF,
    "else",          ELSE,
    "&&",            AND,
    "||",           OR,
    "==",           EQ,
    "!=",          NEQ,
    "<=",          LEQ,
    ">=",          GEQ,
    "for",          FOR,
    "inc",          INC,
    "dec",          DEC,
    "switch",       SWITCH,
    "case",         CASE,
    "default",      DEFAULT,
    "break",        BREAK,
    "void",         VOID,

```

```

    "return",      RETURN,
    NULL,          0           // para marcar el fin de la tabla
} ;

t_keyword *search_keyword (char *symbol_name)
{
    // Busca n_s en la tabla de pal. res.
    // y devuelve puntero a registro (simbolo)

    int i ;
    t_keyword *sim ;

    i = 0 ;
    sim = keywords ;
    while (sim [i].name != NULL) {
        if (strcmp (sim [i].name, symbol_name) == 0) {
            // strcmp(a, b) devuelve == 0 si a==b
            return &(sim [i]) ;
        }
        i++ ;
    }

    return NULL ;
}

```

```

/*****/
/*****/ Tabla de variables locales *****/
/*****/
#define MAX_LOCALS 100
char *local_vars[MAX_LOCALS];
int num_locals = 0;

```

```

void add_local_var(char *name) {
    if (num_locals < MAX_LOCALS) {
        local_vars[num_locals] = gen_code(name);
        num_locals++;
    } else {
        printf("Error: Demasiadas variables locales\n");
    }
}

int is_local_var(char *name) {
    for (int i = 0; i < num_locals; i++) {
        if(strcmp(local_vars[i], name) == 0) {
            return 1;
        }
    }
    return 0;
}

void limpia_vars_locales() {
    num_locals = 0;
}

char *get_var_name(char *name) {
    // Si var está en tabla local, añadimos el prefijo
    if (is_local_var(name)) {
        char temp_name[1024];
        // Dependiendo de la función actual --> cambia el nombre de la variable local
        sprintf(temp_name, "%s_%s", funcion_actual, name);
        return gen_code(temp_name);
    }
    // Parámetro formal, lisp lo gestiona como local, sin prefijo

```



```

    if (is_param(name)) {
        return name;
    }
    // Si var es global (no está en la tabla local), se devuelve tal cual
    return name;
}

/*****/
/*****/ Tabla de parámetros formales *****/
/*****/
#define MAX_PARAMS 50
char *param_vars[MAX_PARAMS];
int num_params = 0;

void add_param(char *name) {
    if (num_params < MAX_PARAMS) {
        param_vars[num_params] = gen_code(name);
        num_params++;
    } else {
        printf("Error: Demasiados parámetros\n");
    }
}

int is_param(char *name) {
    for (int i = 0; i < num_params; i++) {
        if(strcmp(param_vars[i], name) == 0) {
            return 1;
        }
    }
    return 0;
}

```

```
}
```

```
void limpia_params() {  
    num_params = 0;  
}
```

```
/*  
***** Seccion del Analizador Lexicografico *****  
*/
```

```
char *gen_code (char *name)    // copia el argumento a un  
{                               // string en memoria dinamica  
    char *p ;  
    int l ;  
  
    l = strlen (name)+1 ;  
    p = (char *) mi_malloc (l) ;  
    strcpy (p, name) ;  
  
    return p ;  
}
```

```
int yylex ()  
{  
// NO MODIFICAR ESTA FUNCION SIN PERMISO  
    int i ;  
    unsigned char c ;  
    unsigned char cc ;  
    char ops_expandibles [] = "!<=|>%&/+-*" ;
```

```

char temp_str [256] ;
t_keyword *symbol ;

do {
    c = getchar () ;

    if (c == '#') { // Ignora las lineas que empiezan por # (#define, #include)
        do {          // OJO que puede funcionar mal si una linea contiene #
            c = getchar () ;
        } while (c != '\n') ;
    }

    if (c == '/') { // Si la linea contiene un / puede ser inicio de comentario
        cc = getchar () ;
        if (cc != '/') { // Si el siguiente char es / es un comentario, pero...
            ungetc (cc, stdin) ;
        } else {
            c = getchar () ; // ...
            if (c == '@') { // Si es la secuencia //@ ==> transcribimos la linea
                do {          // Se trata de codigo inline (Codigo embebido en C)
                    c = getchar () ;
                    putchar (c) ;
                } while (c != '\n') ;
            } else {          // ==> comentario, ignorar la linea
                while (c != '\n') {
                    c = getchar () ;
                }
            }
        }
    }
} else if (c == '\\') c = getchar () ;

```

```

    if (c == '\n')
        n_line++ ;

} while (c == ' ' || c == '\n' || c == 10 || c == 13 || c == '\t') ;

if (c == '\"') {
    i = 0 ;
    do {
        c = getchar () ;
        temp_str [i++] = c ;
    } while (c != '\"' && i < 255) ;
    if (i == 256) {
        printf ("AVISO: string con mas de 255 caracteres en linea %d\n", n_line) ;
    }
    // habria que leer hasta el siguiente " , pero, y si falta?
    temp_str [--i] = '\0' ;
    yylval.code = gen_code (temp_str) ;
    return (STRING) ;
}

if (c == '.' || (c >= '0' && c <= '9')) {
    ungetc (c, stdin) ;
    scanf ("%d", &yylval.value) ;
//    printf ("\nDEV: NUMBER %d\n", yylval.value) ;           // PARA DEPURAR
    return NUMBER ;
}

if ((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z')) {
    i = 0 ;
    while (((c >= 'A' && c <= 'Z') || (c >= 'a' && c <= 'z') ||
        (c >= '0' && c <= '9') || c == '_') && i < 255) {
        temp_str [i++] = tolower (c) ;
    }
}

```

```

        c = getchar () ;
    }
    temp_str [i] = '\0' ;
    ungetc (c, stdin) ;

    yylval.code = gen_code (temp_str) ;
    symbol = search_keyword (yylval.code) ;
    if (symbol == NULL) {      // no es palabra reservada -> identificador antes vrvariable
//        printf ("\nDEV: IDENTIF %s\n", yylval.code) ;      // PARA DEPURAR
        return (IDENTIF) ;
    } else {
//        printf ("\nDEV: OTRO %s\n", yylval.code) ;          // PARA DEPURAR
        return (symbol->token) ;
    }
}

if (strchr (ops_expandibles, c) != NULL) { // busca c en ops_expandibles
    cc = getchar () ;
    sprintf (temp_str, "%c%c", (char) c, (char) cc) ;
    symbol = search_keyword (temp_str) ;
    if (symbol == NULL) {
        ungetc (cc, stdin) ;
        yylval.code = NULL ;
        return (c) ;
    } else {
        yylval.code = gen_code (temp_str) ; // aunque no se use
        return (symbol->token) ;
    }
}

//    printf ("\nDEV: LITERAL %d %#c#\n", (int) c, c) ;      // PARA DEPURAR

```

```
    if (c == EOF || c == 255 || c == 26) {  
//        printf ("tEOF ") ;                // PARA DEPURAR  
        return (0) ;  
    }  
  
    return c ;  
}
```

```
int main ()  
{  
    yyparse () ;  
}
```